ClawTeams 完整项目需求文档 v2.0

一、项目定位与核心洞察

1.1 产品名称与域名

- 产品名称：**ClawTeams**
- 域名：**clawteams.io**
- 底层技术平台：**OpenClaw**

1.2 核心洞察

会议是上下文对齐失败的症状，不是解决方案。

每个团队都在用会议救火——上下文漂移，开会对齐，继续漂移，再开会。这个循环永远不会自己结束，因为没有任何机制阻止上下文漂移。

ClawTeams从根本上消除上下文漂移，让大多数会议变得不再必要。

1.3 系统定位

ClawTeams是一个**团队协作操作系统**，不是：

- ❌ 项目管理工具
- ❌ AI工具平台
- ❌ 协作软件
- ❌ 自动化工具（Zapier级别）

而是：**让团队意图和执行现实保持持续对齐的操作系统。**

1.4 解决的三个根本问题

① 上下文从未真正对齐 每个人都在用自己理解的目标做事，这个理解从一开始就是碎片化的，随时间推移持续偏离。

② 意图只能自上而下压，无法自下而上反馈 执行层的真实状态无法及时、完整地回到决策层。领导看到的永远是滞后的、过滤过的现实。

③ **没有动态对齐机制** 意图和执行现实之间的偏差靠人工汇报才能发现，发现时往往已经太晚。

1.5 核心设计哲学

- 用户的工作是判断，AI的工作是准备
- 对话是草稿，确认才是提交
- 能力属于角色，贡献属于个人
- 权限从负责关系自动推导，不需要手动配置
- 状态属于团队，执行属于个人
- 执行层记录做了什么，认知层记录学到了什么
- 最好的权限系统是隐形的

二、系统架构总览

2.1 三层架构

```
团队大脑（云端后端）
├── 账号模块
├── 状态模块（意图图谱 + 状态机）
├── 认知模块
├── 权限模块
└── 呈现模块
          ↕
事件总线（神经系统）
          ↕
龙虾个体层（分布式执行末梢）
└── 适配层（Output Hook + Context Injector + Event Subscriber）
└── 本地或云端执行容器
```

2.2 技术选型

组件	技术选型	原因
工作流引擎	Temporal	持久化执行、长时运行、历史回溯、Signal机制
意图图谱	Neo4j或图数据库	节点关系、版本化、语义结构

事件总线	Kafka或WebSocket pub/sub	实时事件驱动
对象存储	Cloudflare R2（早期）/ MinIO（私有化）	R2无出口流量费，MinIO兼容S3接口
镜像仓库	GitHub Container Registry	免费额度，代码仓库集成
龙虾运行环境	Docker容器标准化	跨机器迁移，一键恢复
后端部署	单体后端，模块内部分离	早期不过度拆分，瓶颈时再拆

2.3 三层之间的因果关系

```
共享上下文层（决定理解）
        ↓
能力网络层（决定能做什么）
        ↓
异步协作层（决定怎么做）
```

没有共享上下文 → 能力无法正确调用 没有能力网络 → 协作无法执行 没有异步机制 → 系统无法规模化

三、龙虾个体层

3.1 龙虾的三种形态

形态一：真人龙虾

- 绑定一个真实用户
- 用户不在线时龙虾继续运行
- 用户回来时龙虾汇报状态
- 龙虾不替代这个人，而是这个人在系统里的持续存在形式

形态二：纯AI龙虾

- 无对应真人
- 完全自动执行节点
- 任何人都可以调用

- 例：专门负责信息搜集的龙虾

形态三：真人直接响应

- 系统把子任务路由给真人
- 真人的响应被当成龙虾的输出处理
- 用于必须真人亲自处理的决策性任务

3.2 龙虾账号（Agent Identity）

```
{
  "agent_id": "唯一标识，永久不变",
  "role_id": "所属角色（非人名）",
  "owner_id": "绑定用户，可为空",
  "team_id": "所属团队",
  "capability_scope": ["可调用的能力列表"],
  "trust_level": "信任等级",
  "created_at": "创建时间",
  "state_snapshot_ref": "最新状态快照引用"
}
```

3.3 龙虾适配层（不修改龙虾本身）

① 输出拦截器（Output Hook）

- 捕获龙虾执行输出
- 转换为标准化状态对象
- 写入团队大脑
- 龙虾本身无感知

② 上下文注入器（Context Injector）

- 任务开始前从团队大脑拉取相关上下文
- 注入到龙虾输入前缀
- 龙虾带着团队上下文工作但无需感知

③ 事件订阅器（Event Subscriber）

- 订阅与当前任务相关的事件
- 收到新事件时触发龙虾重新评估执行计划

3.4 本地部署规则

- 龙虾可以在本地电脑运行
- 所有状态实时同步至团队大脑，本地不保留独占数据
- 本地仅作为执行容器（算力），不作为数据归宿
- 网络断开时允许本地临时缓存，重新联网后强制同步
- 使用Docker标准化运行环境

3.5 人员离职流程

```
人类账号注销
→ 龙虾解绑 owner_id
→ 龙虾归属 team_id
→ 团队管理员指定新 owner_id
→ 新人登录，重新授权
→ 龙虾历史上下文、能力配置完整保留
```

四、团队大脑

4.1 账号模块

人类账号（User Account）

- 标准JWT登录体系
- 有会话过期，人不在线即失效

龙虾账号（Agent Identity）

- 长期有效API Key机制
- 7×24持续存在，不依赖人类登录态

账号绑定关系

- 一个人可拥有多个龙虾账号
- 可授权龙虾代表自己行动
- 可设定龙虾的能力边界

- 可随时撤销授权

人类账号与龙虾账号的本质区别

	人类账号	龙虾账号
存在方式	登录验证，有会话	持续运行，无会话
在线要求	人在线才活	7×24存在
权限归属	个人权限	角色权限

4.2 状态模块

状态机本质

团队大脑是一个**带时间维度的状态机**：

- 状态持续演化
- 历史不可篡改，只能追加
- 回滚通过移动状态指针实现，不删除历史

四个子模块

Intent Graph（意图图）

- 当前激活目标（active goals）
- 子任务分解树（task tree）
- 优先级与依赖关系

State Timeline（状态时间线）

- 谁做了什么（actions log）
- 当前进度（progress states）
- 变化差异记录（diff log）

Knowledge Memory（知识沉淀）

- 已验证信息
- 决策依据

- 历史结论与经验

Social Context（关系上下文）

- 角色与负责范围
- 信任权重与authority
- 协作关系图谱

状态写入规则

- 自动写入，不依赖人工维护
- 强语义结构，不接受纯文本
- 版本化存储，支持回滚
- 事件驱动，支持订阅

回滚机制

单龙虾回滚

```
大脑下发指令 → 龙虾恢复指定时间点状态 → 重新执行
```

团队整体回滚

```
大脑协调所有相关龙虾 → 同步回滚到同一时间点 → 保持状态一致性
```

4.3 工作流生成器（核心自研组件）

工作流生成器是一个**编译器**——把意图图谱（声明式"想要什么"）编译成Temporal（命令式"怎么做"）。

四个子模块

① 图谱解析器（Graph Parser）

识别节点类型：

- 目标节点（Goal）
- 任务节点（Task）
- 决策节点（Decision）

- 人工节点（Human）

识别边类型：

- 顺序依赖（A完成后才能做B）
- 并行关系（A和B同时进行）
- 条件依赖（满足条件X才触发B）
- 聚合关系（A和B都完成才触发C）

② 执行策略引擎（Execution Planner）

- 能力匹配：每个任务需要什么能力，哪个龙虾能承接
- 执行路径优化：哪些任务可以并行，关键路径是什么
- 风险识别：哪些节点是瓶颈，哪些失败会阻塞全局

③ Temporal代码生成器（Workflow Compiler）

翻译规则：

```
图谱节点        → Temporal Activity
顺序依赖        → await activity
并行关系        → Promise.all / asyncio.gather
条件依赖        → if/else分支
聚合关系        → 等待所有完成
人工节点        → Temporal Signal（工作流暂停等待）
```

④ 变化监听器（Change Listener）

```
微小变化（优先级调整）→ 动态更新Activity参数，不中断执行
中等变化（新增依赖）   → 在下一个检查点注入
重大变化（核心意图）   → 暂停工作流，重新生成，人工确认后生效
```

AI驱动的完整链路

```
用户自然语言输入
→ AI解析意图，输出结构化JSON（不是自然语言）
→ AI生成执行计划（能力匹配 + 路径优化）
→ AI生成Temporal代码
→ 验证层（静态分析 + Schema验证 + 沙箱试跑）
→ Temporal执行引擎
```

用户永远不需要学习Temporal，不需要懂工作流编排，不需要写任何代码。

意图变化的缓冲机制

```
意图图谱（高频变化）
          ↓
意图暂存区（缓冲）
└── 变化合并：10分钟内多次变化合并为一次
└── 变化分级：微小/中等/重大
└── 检查点机制：在任务自然边界处消化变化
          ↓
工作流（稳定执行）
```

检查点设置在：任务完成时、进入等待状态时、人工审批前、里程碑节点。

若意图变化速度远超检查点频率，系统主动提示：**意图尚未稳定，建议先稳定意图再继续执行。**

4.4 权限模块

权限设计原则

- 权限按**负责范围**划分，不按层级划分
- 负责某个范围的人，对该范围的所有层次都有完整参与权
- 权限从负责关系自动推导，无需手动配置
- 最好的权限系统是隐形的

自动推导规则

```
你负责这个范围
→ 对该范围：完整读写权限（意图到执行所有层次）
→ 对上层：只读权限
→ 对平级：有限只读权限
→ 对下层：完整管理权限
```

层次锁定机制

```
愿景层      → 最高权限 + 二次确认才能修改
战略目标层 → 需明确表达修改意图才触发
项目层      → 较灵活，影响下游时需确认
执行层      → 最自由，随时可调整
```

三种例外需人工配置

1. 跨范围协作
2. 临时授权
3. 敏感内容保护

三种任务节点的权限实现

纯龙虾节点

```
# 检查capability_scope → 注入上下文 → 执行 → 写回状态
```

人工节点（Temporal Signal机制）

```
# 查询approver_role → 推送通知 → 工作流暂停等待Signal
# 超时处理：自动通过 或 升级给上级
```

混合节点

```
# 龙虾先执行生成草稿 → 推送给审批人 → 等待Signal
# 超时自动通过（可配置）
```

每个节点携带的权限参数：

```
class TaskInput:
    required_capability: str   # 需要什么龙虾能力
    agent_id: str              # 指定哪个龙虾
    permission_level: str      # 对应Brain权限层级
    approver_role: str         # 需要什么角色审批
    timeout_hours: int         # 等待人工响应超时
    auto_approve: bool         # 超时是否自动通过
    escalation_user_id: str    # 超时升级给谁
```

4.5 认知模块

设计原则

团队协作本身是集体认知的提升过程。任何层的变化都不只是状态更新，而是一次**认知事件**。

认知节点与执行节点的区别

```
执行节点 → 描述要做什么
认知节点 → 描述为什么这么做、学到了什么
```

两者互相关联，形成完整因果链：

```
假设节点 → 驱动 → 执行节点
执行节点 → 产生 → 认知节点
认知节点 → 修正 → 假设节点
```

认知节点的触发条件

只有一种本质：预期和现实之间出现了无法用执行误差解释的偏差。

具体触发点：

① 假设被现实推翻 不是执行慢了、出了bug，而是"我们以为世界是X，结果世界是Y"。

② 同类执行失败反复出现 单次失败是意外，同类失败三次以上是认知问题不是执行问题。

③ 意图被人工推翻 人工审批时否定方向而不是纠正执行错误，说明意图层的某个前提假设需要重新审视。

认知节点的触发判断

```
偏差在容忍范围内 → 状态机记录，跳过认知层
偏差超出容忍范围 → AI直接写入认知节点（不需要人工确认）
                    → 通知相关人："系统记录了一个新认知，点击查看"
                    → 人觉得不对 → 可以删除或修改
                    → 人没反应 → 默认成立，永久保留
```

关键原则：AI直接写入，人工只有否决权。

与草稿层逻辑相反：

```
意图变化 → 草稿层 → 人工确认才生效（怕破坏结构）
认知节点 → 直接写入 → 人工可以否决（怕漏掉认知）
```

容忍范围由团队自己设定，不是系统决定。

认知阶段枚举

```
假设（Hypothesis）      → 我们相信X会发生
求证中（Testing）       → 正在验证X
```

```
已验证（Validated）     → X被证明是对的
已推翻（Invalidated）  → X被证明是错的
部分成立（Partial）     → X在某些条件下成立
认知迭代（Evolved）     → 因为X，我们对Y的理解改变了
```

四种变化对应四种认知事件

变化来源	认知记录内容
意图层主动变化	为什么改变、原假设、新假设、认知阶段
工作流层变化	原路径为何走不通、障碍是新信息还是执行失误
龙虾执行层变化	预期vs实际、差距是局部还是系统性问题
人工干预	龙虾判断哪里错了、是个例还是模式

长期价值

- 短期：每个决策都有完整认知依据，不重复踩坑
- 中期：团队认知模式变得可见，哪类假设经常被推翻
- 长期：团队集体智慧完整沉淀，新人可快速理解认知演化历史

4.6 呈现模块

三层界面结构

第一层：对话框（输入和判断） 第二层：动态地图（执行现实的实时镜像） 第三层：卡片详情（单个任务的完整因果链）

主界面布局

```
左侧：对话框（输入和判断）
右侧：实时意图地图（执行现实）

对话和地图必须在同一界面，不是两个页面。
用户说话的同时，右侧地图实时响应。
```

动态地图的视觉反馈层次

生成层（AI在工作）

- 节点从模糊到清晰
- 用户可以在此阶段打断和修正

编排层（计划在形成）

- 边和依赖关系动态连接
- 用户可以拖拽调整依赖关系

执行层（龙虾在跑）

```
灰色   → 等待
蓝色   → 执行中
绿色   → 完成
红色   → 失败或阻塞
黄色   → 等待人工介入
橙色虚线框 → 草稿（待确认）
```

认知层（分隔线以下）

```
橙色虚线框 → 假设求证中
红色实线框 → 已推翻
绿色实线框 → 已验证
紫色虚线框 → 认知迭代中
```

地图节点操作

每个节点三个操作：✓ 确认 / ✎ 调整 / ✕ 删除

Temporal状态实时映射

```
Temporal Activity状态变化
→ 触发事件写入事件总线
→ 前端订阅事件流（WebSocket）
→ 实时更新地图节点状态
```

用户看到的不是Temporal，是意图地图的实时演化。

自然语言保护机制

```
自然语言输入
→ AI判断输入类型：
  ├── 新增内容        → 在现有结构上添加节点
```

```
    ├── 局部调整        → 只修改相关节点
    ├── 结构性变化      → 展示影响范围预览，需确认
    └── 查询/闲聊       → 不触发任何修改
→ 修改先进入草稿层
→ 用户确认后合并进正式结构
```

影响范围预览：

```
你的这个调整会影响：
- N个项目的时间线
- N个部门的任务分配
- N个已确认的里程碑
确认修改？ [是] [取消]
```

领导第一次使用流程

```
Step 1：愿景输入（自然语言，约5分钟）
→ 系统引导，领导自由表达
→ AI结构化，领导确认

Step 2：AI自动生成第一版地图（非空白画布）
→ 推导战略目标（3-5个）
→ 推导功能板块
→ 推导第一阶段时间线

Step 3：领导在地图上做判断（不是填写）
→ 逐节点确认/调整/删除
→ AI实时更新地图

Step 4：向下传递（自动）
→ 相关板块推送给负责人
→ 负责人看到AI建议的任务分解
→ 负责人确认/调整后继续向下传递

Step 5：执行层启动
→ 龙虾开始接收任务
→ 适配层注入上下文
→ 执行结果实时回写
```

空白画布是最差的用户体验。第一版地图必须由AI生成，领导只做判断。

五、事件总线

5.1 职责

- 龙虾与团队大脑之间的状态同步
- 龙虾与龙虾之间的直接协同
- 人工介入点的通知和响应
- 团队大脑内部各模块的状态变化传递

5.2 标准事件结构

```
{
  "event_id": "唯一标识",
  "event_type": "task_completed | state_changed | human_required | cognitive_trig
  "source_agent_id": "发起者",
  "team_id": "所属团队",
  "task_id": "关联任务",
  "payload": "结构化数据（非自然语言）",
  "timestamp": "时间戳",
  "requires_human": true/false
}
```

5.3 龙虾↔大脑通信协议（开源部分）

MCP存在于龙虾内部（龙虾用MCP连接工具），但龙虾和团队大脑之间需要独立定义的协议，因为团队大脑是状态机图谱系统，不是LLM。

协议需定义三件事：

注册协议

```
{
  "agent_id": "...",
  "capability_schema": "结构化能力声明（非自然语言）",
  "output_schema": "输出格式定义",
  "subscription_topics": ["订阅的事件类型"]
}
```

状态上报协议

```
{
  "task_id": "...",
  "result": "结构化输出",
  "state": "枚举值：completed/failed/blocked/human_required",
  "requires_human": false,
```

```
  "cognitive_signal": "是否触发认知层判断"
}
```

事件订阅协议

```
{
  "subscribe_topics": ["关心的事件类型"],
  "response_timeout": "秒",
  "timeout_escalation": "超时处理规则"
}
```

能力声明必须是结构化枚举值，不接受自然语言描述。 这是工作流生成器能力匹配准确的前提。

六、对话层

6.1 核心设计原则

对话不能发生在飞书、Slack、Telegram等外部工具里，因为：

```
对话 = 决策过程的实时记录
决策过程 = 认知节点的来源
认知节点 = 团队大脑的核心资产
```

外部工具让大脑系统对决策过程完全盲视。**ClawTeams必须有自己的对话层。**

6.2 三种对话形态

① 人 ↔ 大脑

- 用户和ClawTeams系统对话
- 发指令、查状态、问进度
- 大脑响应，龙虾执行
- 意图最清晰，直接执行

② 两人对话

- 两个人在讨论一件事
- 大脑监听，识别意图和认知
- 适时出现操作按钮，不打断对话节奏

- 需判断谁的意图算数

③ 多人群组

- 多人同时参与
- 需判断共识还是分歧
- 出现分歧时不随便选一方意见执行
- 触发「发起决策」节点，等有权限的人拍板后再执行

6.3 对话气泡的特殊设计

AI实时判断每句话的语义类型，按需出现操作按钮：

```
普通消息        → 普通气泡，不带任何按钮
意图/决策       → 出现「转为执行 →」按钮
认知/假设       → 出现「记入认知层 →」按钮
两者都有        → 两个按钮都出现
多人分歧        → 出现「发起决策 →」按钮
```

按钮出现是AI主动的，点击是人主动的。不强制，不打断，但机会永远在那里。

6.4 气泡状态卡片（点击转为执行后）

气泡原地变形为活的状态卡片，实时反映执行进度：

```
点击前：
「我想关掉几个效果差的ad」
［转为执行 →］［记入认知层 →］

↓ 点击「转为执行」

执行中：
┌──────────────────────────┐
│ 关掉效果差的广告              │
│ ──────────────────────── │
│ 🔵 龙虾拉取广告数据中…        │
└──────────────────────────┘

↓ 数据回来

待确认：
┌──────────────────────────┐
│ 关掉效果差的广告              │
│ ──────────────────────── │
```

```
│ ✓ 找到4个CTR低于0.8%      │
│ [查看名单] [确认关闭]      │
└──────────────────────────┘

↓ 确认后

完成:

┌──────────────────────────┐
│ 关掉效果差的广告      ✓    │
│ ──────────────────────── │
│ 已关闭4个广告              │
│ 3天后自动回来验证CTR变化   │
│ [查看详情]                 │
└──────────────────────────┘

↓ 3天后龙虾自动追加

验证结果:

┌──────────────────────────┐
│ CTR变化验证                │
│ ──────────────────────── │
│ 整体CTR +23%               │
│ 假设「ad过多分散budget」   │
│ → 已验证 ✓                 │
└──────────────────────────┘
```

对话记录本身就是任务的完整时间轴。 从决策对话到执行结果到认知验证，全部发生在同一条消息线上。

七、任务路由系统

7.1 路由判断逻辑（OODA Loop）

```
Observe → 龙虾实时采集执行数据，写入团队大脑
Orient  → AI整理上下文，比对目标与现状
Decide  → 人在自己负责的范围内做判断
Act     → 路由给龙虾或人执行
→ 回到 Observe
```

系统的核心竞争力：让团队的OODA Loop转速远超传统协作方式。

7.2 路由决策树

```
收到任务
↓
纯AI可以完成？ → 路由给纯AI龙虾，全自动执行
↓ 否
需要某人风格判断？ → 路由给真人龙虾，AI先执行，真人审批
↓ 否
必须真人亲自做？ → 推送通知给真人，等待Signal
↓ 阻塞超时
升级给管理员处理
```

7.3 前端状态可视化

```
纯龙虾节点   → 蓝色自动流转
人工节点     → 黄色高亮 + 推送通知
混合节点     → 蓝色执行完变黄色等待审批
权限不匹配   → 红色 + 说明需要谁处理
超时升级     → 节点闪烁 + 通知升级对象
```

八、档案系统

8.1 档案系统与状态机的区别

```
状态机 → 存过程（轻量、结构化、高频读写）
档案系统 → 存结果（重量、非结构化、低频读写）
```

两者通过artifact_id关联，各自独立演化。状态机只存引用，不存实体。

8.2 四个子模块

① 工作成果档案（Artifact Store）

```
{
  "artifact_id": "唯一标识",
  "task_id": "来源任务",
  "agent_id": "产出龙虾",
  "team_id": "归属团队",
  "type": "document/code/data/media",
  "content_hash": "内容指纹",
  "storage_url": "实际存储地址",
  "version": "版本号",
  "permissions": "访问权限"
```

```
}
```

② 龙虾本体档案（Agent Registry）

- 龙虾配置（system prompt、参数、工具列表）
- 运行环境（Docker镜像版本、依赖版本）
- 能力声明（capability schema）
- 历史版本（每次配置变更都保留）

③ 运行环境档案（Environment Store）

- Docker镜像私有仓库
- 版本化管理
- 支持一键拉取恢复

④ 公共传输层（Transfer Hub）

大文件流转使用预签名URL机制：

```
龙虾产出大文件
→ 上传到对象存储
→ 生成有时效的预签名URL
→ URL写入事件总线
→ 目标龙虾用URL直接下载
→ URL过期自动失效
```

8.3 版本控制

每次写入追加版本，不覆盖：

```
artifact_id: "report_001"
├── v1: 龙虾A初稿
├── v2: 人工修改后
└── current → v2（指针）
```

回滚时：状态机回滚到t2，档案系统自动将current指针移回v2，两者保持一致。

8.4 存储分层

数据类型	存储方案

结构化数据、JSON	图数据库直存
文档、报告	R2对象存储 + 元数据存图数据库
大文件、数据集	R2 + CDN + 预签名URL
Docker镜像	GitHub Container Registry / Harbor

九、多人协作网络

9.1 师徒关系

一个会用龙虾的人帮另一个人配置龙虾，自然产生：

- **社交关系**：信任链，师傅的推荐有权重
- **技术关系**：师傅龙虾知道徒弟龙虾的配置和能力边界，任务可以在两个龙虾之间流转

9.2 能力图谱涌现

不同师傅的徒弟，通过任务互补自然产生连接：

- 任务反复路由给同样几个龙虾 → 事实上的协作团队自然形成
- 不依赖人和人先认识，依赖任务互补

9.3 龙虾管理员角色

每个团队的超级节点：

- 负责为团队成员配置和维护龙虾
- 其他成员只需发起任务、接收结果
- 所有配置操作自动记录，支持交接

9.4 龙虾出租与记忆隔离

记忆分层设计：

```
龙虾记忆
├── 能力记忆（可跨域流通）
│    └── 任务最优执行路径、踩过的坑、风格偏好
└── 上下文记忆（必须域内隔离）
     └── 具体数据、客户信息、业务内容
```

龙虾进入新域时：

- 能力记忆：随龙虾携带，只读
- 上下文记忆：清空，在新域内重新建立
- 新域产生的上下文记忆：留在新域的Brain里
- 龙虾离开时：上下文记忆不跟走

记忆所有权声明（写入时标注）：

```
{
  "ownership": "agent | tenant",
  "visibility": "cross-domain | domain-only",
  "expiry": "permanent | contract-end"
}
```

注意： 由于龙虾的记忆系统（如OpenClaw内部）不受ClawTeams控制，记忆隔离通过协议层约束（控制进入Brain的数据）和合同层约束（商业协议规定违约责任）来实现。龙虾出租功能短期内适合对安全要求不高的中小团队，企业级客户需等待龙虾记忆可审计后推进。

十、部署形态

10.1 两种部署模式

云端SaaS（中小团队）

- 安全要求较低
- 适合社会化协作模型
- 龙虾可跨团队出租
- ClawTeams云端作为中间层保证协议安全

私有化部署（企业客户）

- 数据合规要求高
- Brain私有化部署
- 龙虾在企业内网运行
- 对象存储换用MinIO（完全兼容S3接口，代码零改动）

两种模式是同一个产品的不同部署形态，不是两个产品。（参考GitLab模式）

十一、开源战略

11.1 开源模式：协议开源 + 核心封闭

开源部分：龙虾↔大脑通信协议

- 注册协议、状态上报协议、事件订阅协议
- 验证工具（测试某个龙虾实现是否符合标准）
- 协议本身的演化（社区提RFC）

封闭部分：团队大脑核心

- Alignment Engine
- 动态路由算法
- 上下文对齐机制
- 云端服务

11.2 开源时机

现阶段： 协议内部先跑通，不急于开源。Agent生态尚未充分分化，开源协议的开发者空间有限。

等待时机： Agent生态开始分化，出现大量不同龙虾系统（LangChain、AutoGen、Dify、各类自研Agent）。

时机成熟后： 开源协议，顺势成为Agent协作的行业标准。协议被广泛采用后，ClawTeams从产品变成**Agent协作的基础设施标准**。

11.3 防止被社区带跑的机制

① RFC机制 任何新功能必须先提RFC，说明如何服务于"上下文对齐"这个核心目标。

② 贡献分层

```
欢迎贡献：协议适配器、验证工具、文档
需要讨论：协议扩展
核心团队决定：大脑逻辑、产品方向
```

③ 强势的产品宣言 README第一行写清楚ClawTeams解决什么、不解决什么。

11.4 参考模型

- **Temporal**：开放工作流引擎协议，云服务收费
- **GitLab**：Open Core，核心开放，企业功能封闭

十二、冷启动路径

```
阶段一：0→1 人工验证（3个团队）
创始团队亲自下场
→ 帮团队配好龙虾
→ 跑第一次多人执行
→ 全程录像作为推广素材
→ 交付结果（不是演示，是真实产出）

阶段二：培养超级节点
每个团队培养1个龙虾管理员
→ 管理员负责团队内部维护
→ 其他人只管使用结果

阶段三：网络扩散
3个管理员各自扩散到新团队
→ 产生真实多人协作数据

阶段四：产品打磨
根据真实数据迭代产品
→ 开始规模化推广
```

冷启动的核心原则：不靠用户自学，靠创始团队亲自下场服务，把第一批结果做出来。

产品的第一个用户不是"会用龙虾的人"，而是"看到龙虾帮他们出了结果的团队"。

十三、技术开发优先级

13.1 工作流生成器开发顺序

```
第一步：图谱解析器
└── 先支持简单顺序和并行，不支持条件

第二步：Temporal代码生成器
└── 把解析结果跑起来，验证基本执行链路

第三步：执行策略引擎
```

```
└── 加入能力匹配和路径优化
└── 龙虾协作才真正成立

第四步：变化监听器
└── 最后做，最复杂
└── 前三步跑通后再处理动态调整
```

13.2 AI驱动链路的验证顺序

```
第一步：只用AI做自然语言→图谱（验证语义理解准确率）
第二步：加入AI执行计划生成（验证能力匹配准确率）
第三步：加入AI代码生成（此时已有足够验证数据）
```

每一步验证AI可靠性后再推进，三层叠加的错误极难调试。

13.3 最小状态单元定义（优先级最高）

一个龙虾完成一次任务之后，它必须留下什么，才能让下一个龙虾（或它自己下次）接得上？

这个问题的答案决定整个系统的地基。 必须在其他模块开发前先回答清楚。

十四、失败点清单

层次	失败点
共享上下文层	❌ 变成Notion/文档工具（死数据）
共享上下文层	❌ 依赖用户主动更新（必死）
共享上下文层	❌ 无结构（LLM无法有效利用）
能力网络层	❌ 只是技能marketplace（太浅）
能力网络层	❌ 没有执行能力（只有描述）
能力网络层	❌ 无法组合（不能形成链）
异步协作层	❌ 变成自动化工具（Zapier级别）
异步协作层	❌ 无人类反馈机制（失控）
异步协作层	❌ 强依赖实时交互（失去异步价值）
认知层	❌ 记录执行数据而非认知数据（和状态机重复）

认知层	❌ 强制人工勾选（依赖人主动维护，必死）
权限系统	❌ 按层级分权限（科层制数字化复制，把系统做死）
对话层	❌ 对话发生在外部工具（大脑盲视）
工作流生成器	❌ 能力声明用自然语言（匹配不准确）

文档版本：v2.0 | 基于ClawTeams全程设计对话整理 | clawteams.io